MTRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

Name of Faculty: MOHIT SHARMA

Subject & Code: ANALYSIS OF ALGORITHM & 6CS4-02

Branch: CSE 6TH SEM | Batch: A&B

Semester: VIth | **Session:** 2023-24(even Sem.)

Unit No.: 3 | **Date of Submission: 15-03-2024**

Lect.No.	Topics Covered	Page No.	Total pages
1	Dynamic programming: Principles of dynamic programming	2-8	7
2	Applications: Rod cutting problem	9-18	10
3	Floyd-Warshall algorithm for all pair shortest paths	19-26	8
4	Matrix multiplication, travelling salesman Problem	27-43	14
5	Longest Common sequence, Back tracking	44-49	6
6	8-queen problem, and Knapsack problem	50-51	2
7	Traveling Salesman problem	52-53	2

References Book:

1. T .H .Cormen, C .E .Leiserson, R .L . Rivest "Introduction to Algorithms", 3rd Ed.,PHI, 2011
2. **E. Horowitz, S. Sahni, and S. Rajsekaran, "Fundamentals of Computer Algorithms,"Galgotia Publication**
3. **S.K Basu- Design Methods and Analysis of Algorithms, 2nd Ed., PHI**

15/3/2024

FACULTY **HOD** **DEAN ACADEMICS**

UNIT 3

Lecture 1

Dynamic Programming (DP)

- **Dynamic Programming (DP)** is defined as a technique that solves some particular type of problems in Polynomial Time. Dynamic Programming solutions are faster than the exponential brute method and can be easily proved their correctness.

Dynamic Programming is mainly an optimization over plain recursion. Wherever we see a recursive solution that has repeated calls for the same inputs, we can optimize it using Dynamic Programming. The idea is to simply store the results of subproblems so that we do not have to re-compute them when needed later. This simple optimization reduces time complexities from exponential to polynomial.

Characteristics of Dynamic Programming Algorithm:

- In general, dynamic programming (DP) is one of the most powerful techniques for solving a certain class of problems.
- There is an elegant way to formulate the approach and a very simple thinking process, and the coding part is very easy.
- Essentially, it is a simple idea, after solving a problem with a given input, save the result as a reference for future use, so you won't have to re-solve it.. briefly 'Remember your Past' :).
- It is a big hint for DP if the given problem can be broken up into smaller sub-problems, and these smaller subproblems can be divided into still smaller ones, and in this process, you see some overlapping subproblems.
- Additionally, the optimal solutions to the subproblems contribute to the optimal solution of the given problem (referred to as the Optimal Substructure Property.
- The solutions to the subproblems are stored in a table or array (memoization) or in a bottom-up manner (tabulation) to avoid redundant computation.
- The solution to the problem can be constructed from the solutions to the subproblems.
- Dynamic programming can be implemented using a recursive algorithm, where the solutions to subproblems are found recursively, or using an iterative algorithm, where the solutions are found by working through the subproblems in a specific order.

Dynamic programming works on following principles:

- Characterize structure of optimal solution, i.e. build a mathematical model of the solution.
- Recursively define the value of the optimal solution.
- Using bottom-up approach, compute the value of the optimal solution for each possible subproblems.
- Construct optimal solution for the original problem using information computed in the previous step.

Applications:

Dynamic programming is used to solve optimization problems. It is used to solve many real-life problems such as,

(i) Make a change problem

(ii) Knapsack problem

(iii) Optimal binary search tree

What is the difference between a Dynamic programming algorithm and recursion?

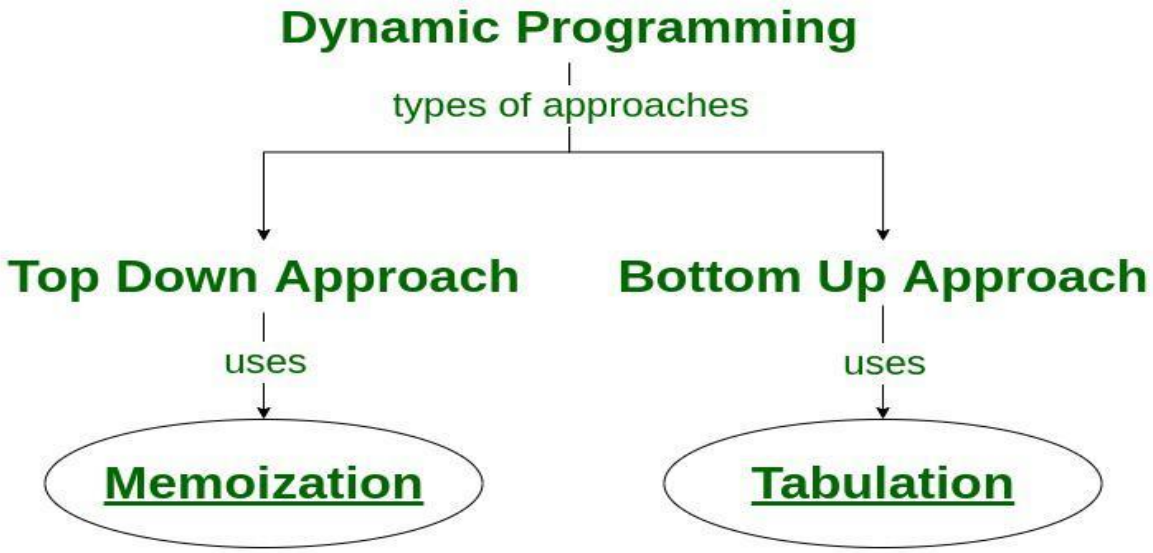
- In **dynamic programming**, problems are solved by breaking them down into smaller ones to solve the larger ones, while **recursion** is when a function is called and executed by itself. While dynamic programming can function without making use of recursion techniques, since the purpose of dynamic programming is to optimize and accelerate the process, programmers usually make use of recursion techniques to accelerate and turn the process efficiently.
- When a function can execute a specific task by calling itself, receive the name of the recursive function. In order to perform and accomplish the work, this function calls itself when it has to be executed.
- Using **dynamic programming**, you can break a problem into smaller parts, called subproblems, to solve it. Dynamic programming involves solving the problem for the first time, then using memoization to store the solutions.
- Therefore, the main difference between the two techniques is their intended use; **recursion** is used to automate a function, whereas dynamic programming is an optimization technique used to solve problems.
- **Recursive** functions recognize when they are needed, execute themselves, then stop working. When the function identifies the moment it is needed, it calls itself and is executed; this is called a recursive case. As a result, the function must stop once the task is completed, known as the base case.
- By establishing states, **dynamic programming** recognizes the problem and divides it into sub-problems in order to solve the whole scene. After solving these sub-problems, or variables, the programmer must establish a

mathematical relationship between them. Last but not least, these solutions and results are stored as algorithms, so they can be accessed in the future without having to solve the whole problem again.

Techniques to solve Dynamic Programming Problems:

1. Top-Down(Memoization):
Break down the given problem in order to begin solving it. If you see that the problem has already been solved, return the saved answer. If it hasn't been solved, solve it and save it. This is usually easy to think of and very intuitive.

2. Bottom-Up(Dynamic Programming):
Analyze the problem and see in what order the subproblems are solved, and work your way up from the trivial subproblem to the given problem. This process ensures that the subproblems are solved before the main problem. This is referred to as Dynamic Programming.



Types of the approach of dynamic programming algorithm

Tabulation(Dynamic Programming) vs Memoization:
There are two different ways to store the values so that the values of a sub-problem can be reused. Here, will discuss two patterns of solving dynamic programming (DP) problems:

- Tabulation: Bottom Up
- Memoization: Top Down

Before getting to the definitions of the above two terms consider the following statements:

- **Version 1**: I will study the theory of DP from GeeksforGeeks, then I will practice some problems on classic DP and hence I will master DP.

- **Version 2**: To Master DP, I would have to practice Dynamic problems and practice problems – Firstly, I would have to study some theories of DP from GeeksforGeeks

Both versions say the same thing, the difference simply lies in the way of conveying the message and that's exactly what Bottom-Up and Top-Down DP do. Version 1 can be related to Bottom-Up DP and Version-2 can be related to Top-Down DP.

	Tabulation	**Memoization**
State	State transition relation is difficult to think	State Transition relation is easy to think
Code	Code gets complicated when a lot of conditions are required	Code is easy and less complicated
Speed	Fast, as we directly access previous states from the table	Slow due to a lot of recursive calls and return statements
Subproblem solving	If all subproblems must be solved at least once, a bottom-up dynamic programming algorithm usually outperforms a top-down memoized algorithm by a constant factor	If some subproblems in the subproblem space need not be solved at all, the memoized solution has the advantage of solving only those subproblems that are definitely required
Table entries	In the Tabulated version, starting from the first entry, all entries are filled one by one	Unlike the Tabulated version, all entries of the lookup table are not necessarily filled in Memoized version. The table is filled on demand.
Approach	Generally, tabulation(dynamic programming) is an iterative	On the other hand, memoization is a

	Tabulation	Memoization
	approach	recursive approach.

Greedy approach vs Dynamic programming

Feature	Greedy method	Dynamic programming
Feasibility	In a greedy Algorithm, we make whatever choice seems best at the moment in the hope that it will lead to global optimal solution.	In Dynamic Programming we make decision at each step considering current problem and solution to previously solved sub problem to calculate optimal solution .
Optimality	In Greedy Method, sometimes there is no such guarantee of getting Optimal Solution.	It is guaranteed that Dynamic Programming will generate an optimal solution as it generally considers all possible cases and then choose the best.
Recursion	A greedy method follows the problem solving	A Dynamic programming is an algorithmic

Feature	Greedy method	Dynamic programming
	heuristic of making the locally optimal choice at each stage.	technique which is usually based on a recurrent formula that uses some previously calculated states.
Memoization	It is more efficient in terms of memory as it never look back or revise previous choices	It requires Dynamic Programming table for Memoization and it increases it's memory complexity.
Time complexity	Greedy methods are generally faster. For example, Dijkstra's shortest path algorithm takes O(ELogV + VLogV) time.	Dynamic Programming is generally slower. For example, Bellman Ford algorithm takes O(VE) time.
Fashion	The greedy method computes its solution by making its choices in a serial forward fashion, never looking back or revising previous choices.	Dynamic programming computes its solution bottom up or top down by synthesizing them from smaller optimal sub solutions.

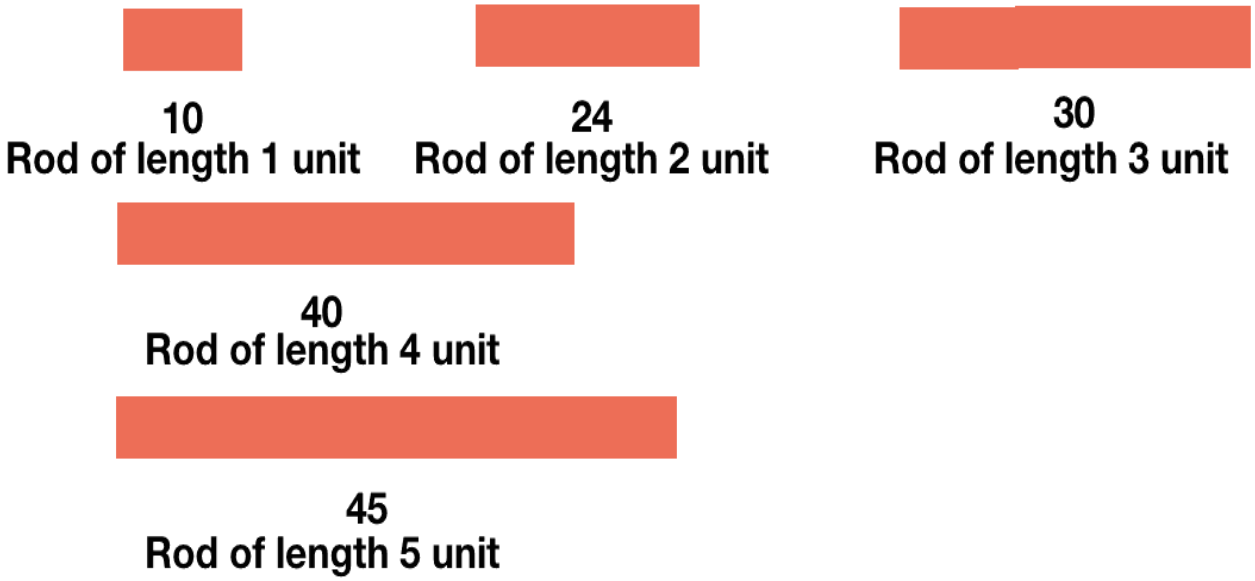
Feature	Greedy method	Dynamic programming
Example	Fractional knapsack .	0/1 knapsack problem

Lecture 2

Rod Cutting | Dynamic Programming

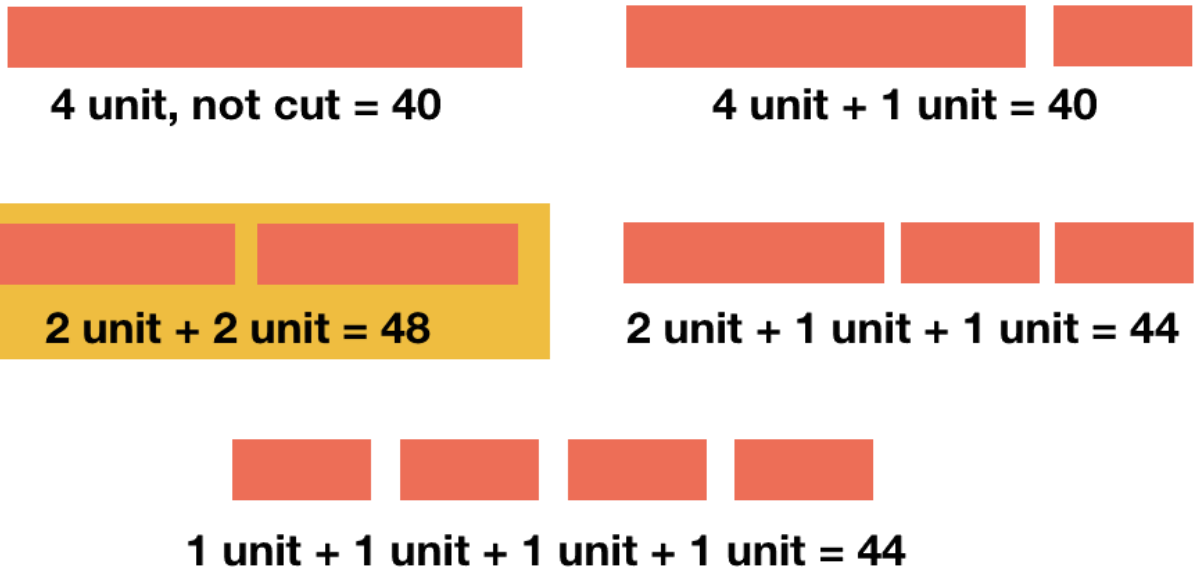
Rod cutting is another kind of problem which can be solved without using dynamic programming approach but we can optimize it greatly by using it. According to the problem, we are provided with a long rod of length n units. We can cut the rod in different sizes and each size has a different cost associated with it i.e., a rod of i units length will have a cost of ci $. Let's look at the table given below showing the cost v/s length of the rod.

Length	1	2	3	4	5
Price	10	24	30	40	45



From the above table, you can see that the rod of length 1 unit has a price of 10$ and a rod of length 2 has a price of 24$, etc. Now, our task is to generate the pieces of the rod in such a way that the revenue generated by selling all the pieces is maximum (let's say this maximum revenue is rn for a rod of length n units).

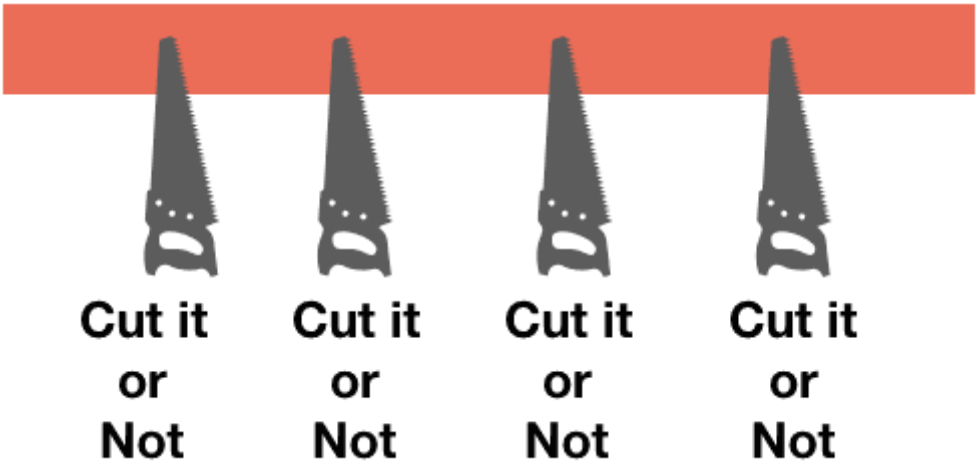
Let's take a case when our rod is 4 units long, then we have the following different ways of cutting it:



Thus, you can see that for a rod of 4 units long, cutting it into two pieces of 2 units length generates the maximum profit for us.

Our task is to find the value of rn (maximum revenue that can be generated from a rod of length n units).
Before moving further, let's first analyze the brute force way of solving it i.e., by checking all the possible solutions.

For a rod of n units long, for every i units, we have two choices - either make that cut or not.

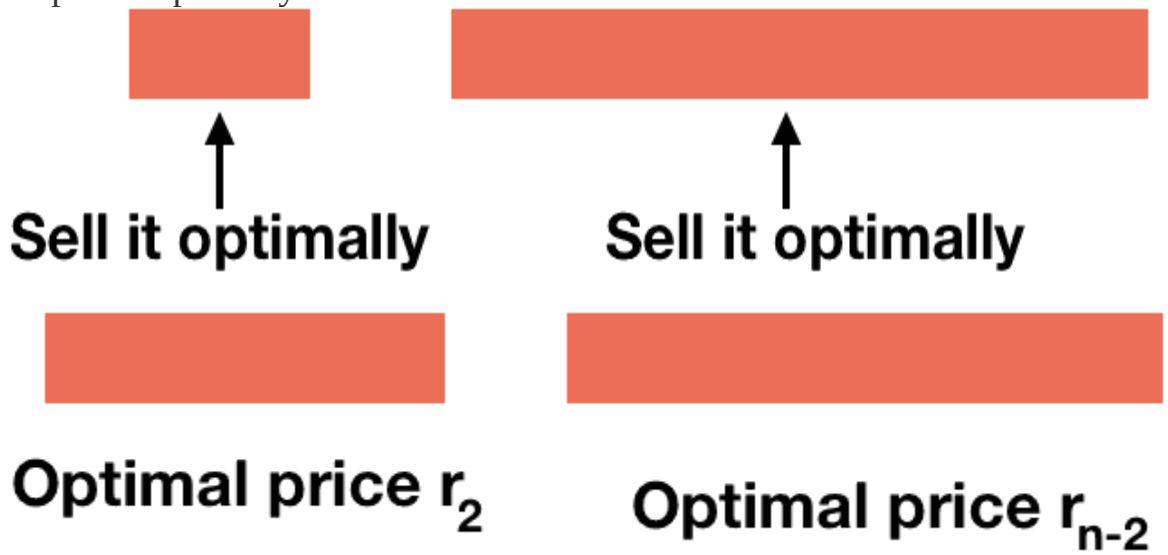


So, we have to choose between two option for a total of n-1 times and thus the total possible number of solutions are $2*2*...*2n-1$ times$=2n-12*2*...*2-1$ times$=2$.
Like the 0-1 knapsack problem, this is also exponential and we need to optimize this to make it useable.

Approach to Solve Rod Cutting Problem

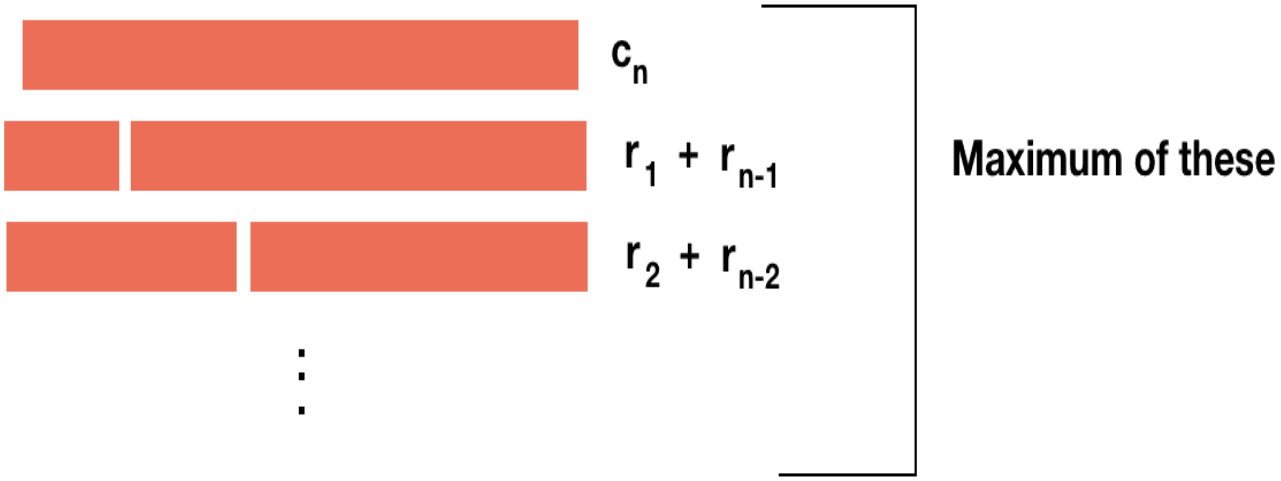
First Approach

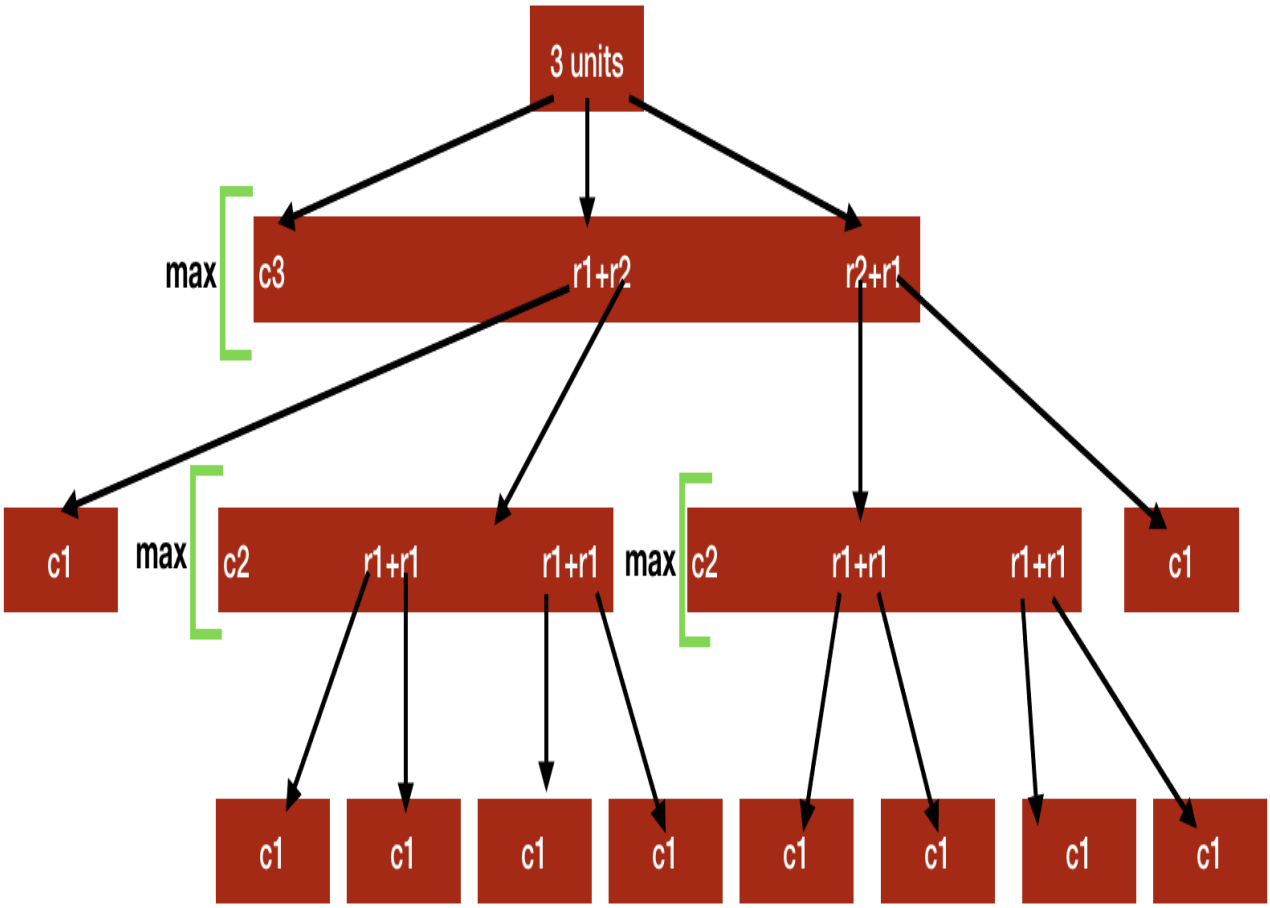
Let's say we have a rod of n units long. Now, we can make cuts at 1, 2, 3, ... n-1 units of length or even no cut at all. In the case of no cutting at all, the rod will be sold at cn $.
Let's say we have made a cut at 1 unit of length. Now, we have two pieces. So we will generate maximum revenue from these two pieces i.e., by selling both the pieces optimally.



You can see that the optimal solution of the problem is incorporating the optimal solutions of the subproblems also. For example, by selling the smaller pieces at the optimal price, we are generating maximum profit from the bigger piece. This property is called **optimal substructure**.

We can say that when making a cut at i unit length, the maximum revenue can be generated by selling the first unit at ri $ and the second unit at rn−i $.
The maximum revenue for a rod of length n (rn will be the maximum of all these revenues.

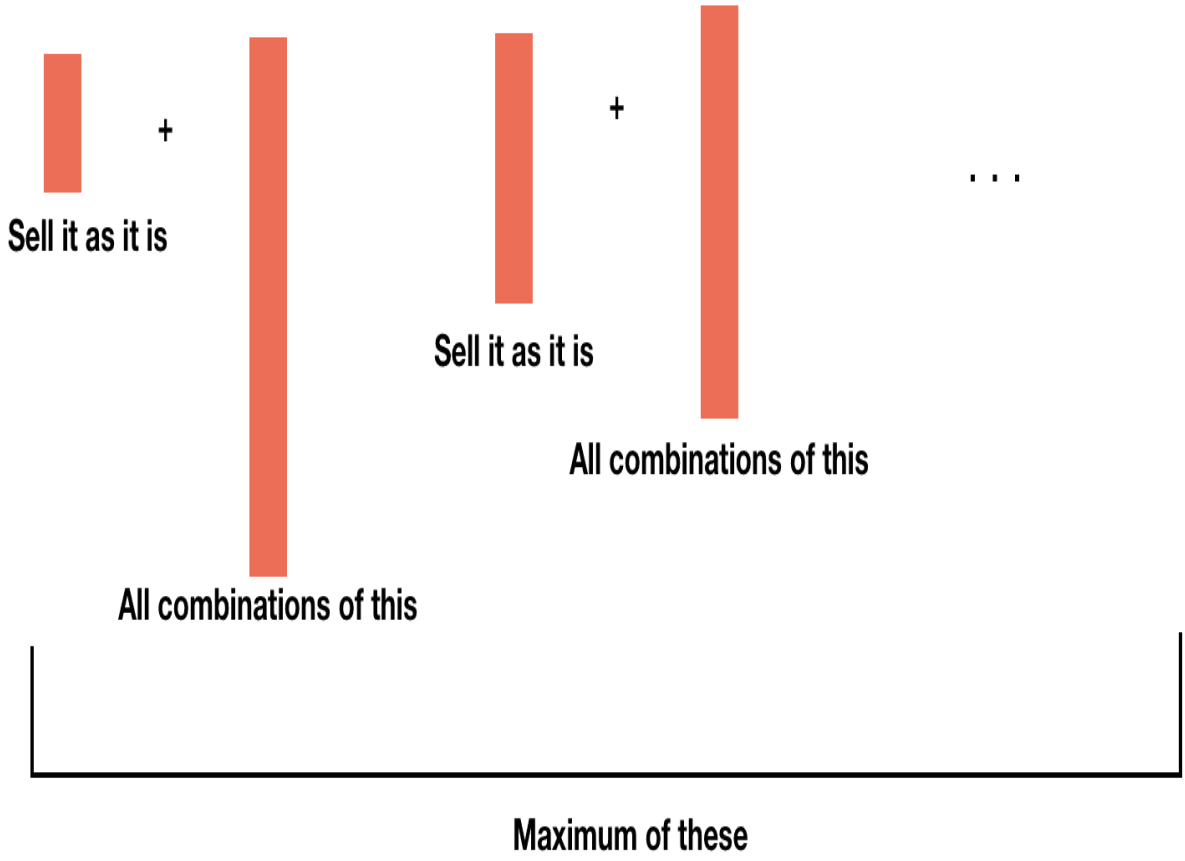




From the above picture, you can see that there are many overlapping subproblems i.e., subproblems are repeated many times. We can reduce significantly by thinking the solution of this problem in a slightly different way.

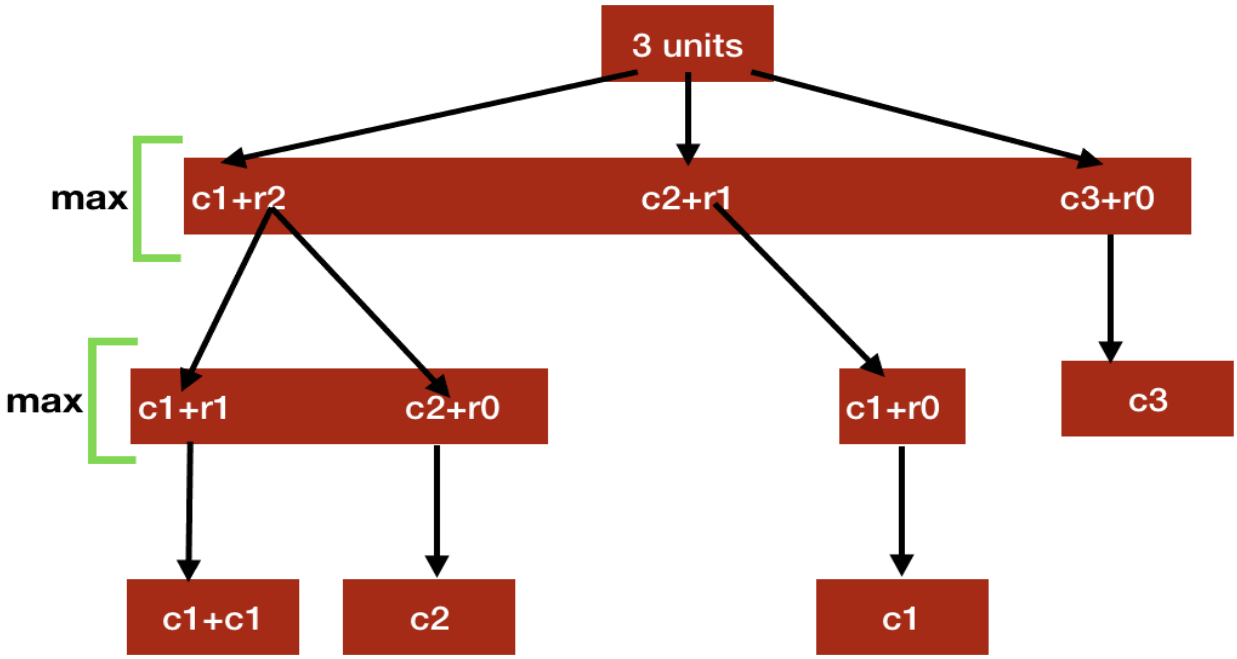
Second Approach

There are different ways in which the rod can be cut. All these different ways can be classified having two parts - one part containing the uncut part of the rod of length i and the other part containing all different possible combination of the rest of the rod (n-i). For example, let's take the case of the first part having the length of 1 unit, then we are going to sell this first piece as it is and the second part will contain all the other combination made by cutting the rod of length n-i differently.



In this case, the maximum revenue generated will be c1+rn−1−1 i.e., selling the first piece uncut and selling the second piece by cutting in a way to get the optimal value.

Next, we can have the rod of length 2 units uncut on the first part and all the other possible combinations made by cutting the rod of length n-2 units differently on the other part.



You can see that we have reduced the number of subproblems by using this formula.

Let's take the price table given above and find the optimal revenue for each length.

Length	1	2	3	4	5
Price	10	24	30	40	45

r1 = 10
r2 = 24
r3 = 34
r4 = 48
r5 = 58

After finding the solution of the problem, let's code the solution.

Code for Rod cutting problem

Let's look at the top-down dynamic programming code first.

Top Down Code for Rod Cutting

We need the cost array (c) and the length of the rod (n) to begin with, so we will start our function with these two - TOP-DOWN-ROD-CUTTING(c, n)

We already know that we are going to use dynamic programming, so we will start by making an array to store the maximum revenue that can be generated by different lengths i.e., r[n+1] so that we don't have to recalculate this value again and again.

Initially, we don't know the value of the maximum revenue that can be generated by these lengths, so we will set all the elements of this array to something negative i.e., $-\infty-\infty$. We can't initialize all the elements with 0 because the maximum revenue generated by the rod of length 0 is 0, so we will make the r[0] = 0.

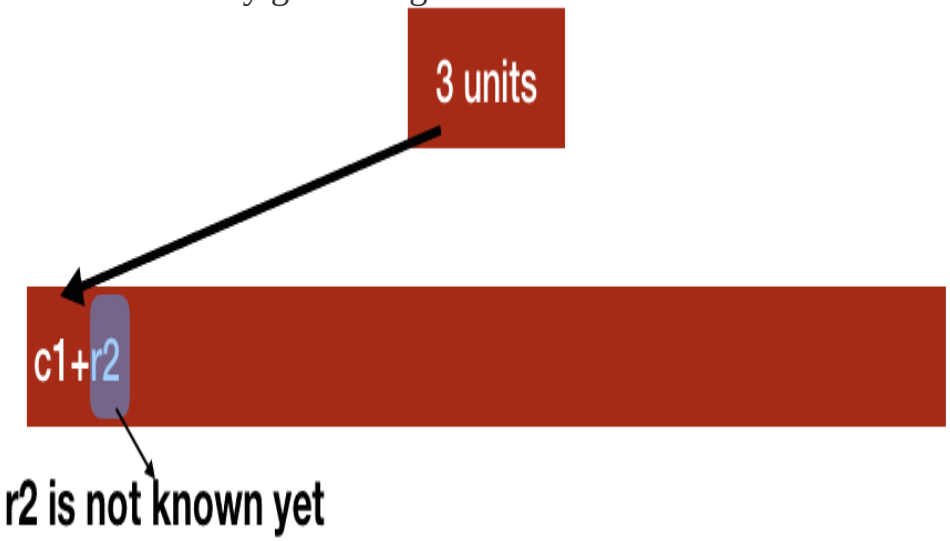
```
r[0] = 0
for i in 1 to n
  r[i] = -INF
```

In the top down approach, we just start solving the problem naturally, so we will just start checking if the there is already a solution stored in the array or not.

```
if r[n] >= 0
  return r[n]
```

If the revenue is not already in the array r, then we will have to calculate it and for that, we need to calculate the maximum value among $(ci+rn-i)$ for i ranging from 1 to n. Basically, $(ci+rn-i)$means the ci + maximum revenue that can be generated by rod of length n-i i.e., TOP-DOWN-ROD-CUTTING(c, n-i). Take note that we can't directly use the array r to get the value of r[n-i] because the

array r doesn't store this value yet or at least we are not sure that it does or doesn't. This would have been in the case of bottom-up implementation in which we start by generating the smaller values first.



Now, the calculation of the maximum of (ci+rn−i) is similar to calculating the sum of all the elements of an array. We will have a variable to store the maximum revenue and then we will iterate from 1 to n and compare the current maximum revenue stored in the variable with the revenue c[i] + TOP-DOWN-ROD-CUTTING(c, n-i).

```
maximum_revenue = -INF
for i in 1 to n
 maximum_revenue = max(maximum_revenue, c[i] + TOP-DOWN-ROD-CUTTING(c, n-i))
```

As stated, we started by having a variable to store the maximum revenue i.e., maximum_revenue = -INF and then we iterate from 1 to n to find the maximum among c[i]+TOP-DOWN-ROD-CUTTING(c, n-i).

Bottom-Up Code for Rod Cutting

In the bottom-up technique, we start by filling the array from start. So, we will first initialize an array r and then we will iterate over it to fill it.

```
r[n+1]
r[0]                    =                    0
for i in 1 to j
```

Now, the array r contains the maximum revenue that can pe generated by each length. For example, the 4th element of the array r contains the maximum revenue that can be generated by a rod of 4 units long. So to calculate it, we need to calculate the maximum of (ci+rn−i) for i ranging from 1 to 4. And thus, we need to iterate again for the different values of i.

```
for i in 1 to j
  max_revenue                    =                    -INF
  for i in 1 to j
    max_revenue = max(max_revenue, c[i] + r[j-i])
  r[j]                    =                    max_revenue
return r[n]
```

Take a note that we can directly use the value stored in the array r for rn−i because we know that we have already filled all the smaller values of the array.

Lecture 3

Floyd-Warshall Algorithm

The Floyd-Warshall algorithm is a dynamic programming algorithm used to discover the shortest paths in a weighted graph, which includes negative weight cycles. The algorithm works with the aid of computing the shortest direction between every pair of vertices within the graph, the usage of a matrix of intermediate vertices to keep music of the exceptional-recognized route thus far.

But before we get started, let us briefly understand what Dynamic Programming is.

Dynamic Programming

Dynamic programming is a technique used in computer science and mathematics to remedy complicated troubles with the aid of breaking them down into smaller subproblems and solving each subproblem as simple as soon as. It is a technique of optimization that can be used to locate the pleasant technique to a hassle with the aid of utilizing the solutions to its subproblems.

The key idea behind dynamic programming is to keep the solutions to the subproblems in memory, so they can be reused later whilst solving larger problems. This reduces the time and area complexity of the set of rules and lets it resolve tons larger and extra complex issues than a brute force approach might.

There are two important styles of dynamic programming:

1. Memoization
2. Tabulation

Memoization involves storing the outcomes of every subproblem in a cache, in order that they may be reused later. Tabulation includes building a desk of answers to subproblems in a bottom-up manner, beginning with the smallest subproblems and working as much as the larger ones. Dynamic programming is utilized in an extensive range of packages, including optimization troubles, computational geometry, gadget studying, and natural language processing.

Some well-known examples of problems that may be solved by the usage of dynamic programming consist of the Fibonacci collection, the Knapsack trouble, and the shortest path problem.

History of Floyd-Warshall algorithm:

The Floyd-Warshall set of rules was advanced independently via Robert Floyd and Stephen Warshall in 1962. Robert Floyd turned into a mathematician and computer scientist at IBM's Thomas J. Watson Research Center, whilst Stephen Warshall became a computer scientist at the University of California, Berkeley. The algorithm was originally developed for use inside the field of operations research, where it turned into used to solve the all-pairs shortest direction problem in directed graphs with tremendous or negative side weights. The problem become of outstanding hobby in operations research, as it has many applications in transportation, conversation, and logistics.

Floyd first presented the set of rules in a technical record titled "**Algorithm 97: Shortest Path" in 1962**. Warshall independently discovered the set of rules quickly afterwards and posted it in his personal technical document, "**A Theorem on Boolean Matrices**". The algorithm has on account that emerged as a cornerstone of pc technology and is broadly used in lots of regions of studies and enterprise. Its capability to correctly find the shortest paths between all pairs of vertices in a graph, including those with terrible side weights, makes it a treasured tool for solving an extensive range of optimization problems.

Working of Floyd-Warshall Algorithm:

The set of rules works as follows:

1. Initialize a distance matrix D wherein D[i][j] represents the shortest distance between vertex i and vertex j.
2. Set the diagonal entries of the matrix to 0, and all other entries to infinity.
3. For every area (u,v) inside the graph, replace the gap matrix to mirror the weight of the brink: D[u][v] = weight(u,v).
4. For every vertex okay in the graph, bear in mind all pairs of vertices (i,j) and check if the path from i to j through k is shorter than the current best path. If it is, update the gap matrix: D[i][j] = min(D[i][j], D[i][k] D[k][j]).
5. After all iterations, the matrix D will contain the shortest course distances between all pairs of vertices.

Example:

Floyd-Warshall is an algorithm used to locate the shortest course between all pairs of vertices in a weighted graph. It works by means of keeping a matrix of distances between each pair of vertices and updating this matrix iteratively till the shortest paths are discovered.

Let's see at an example to illustrate how the Floyd-Warshall algorithm works:

Consider the following weighted graph:

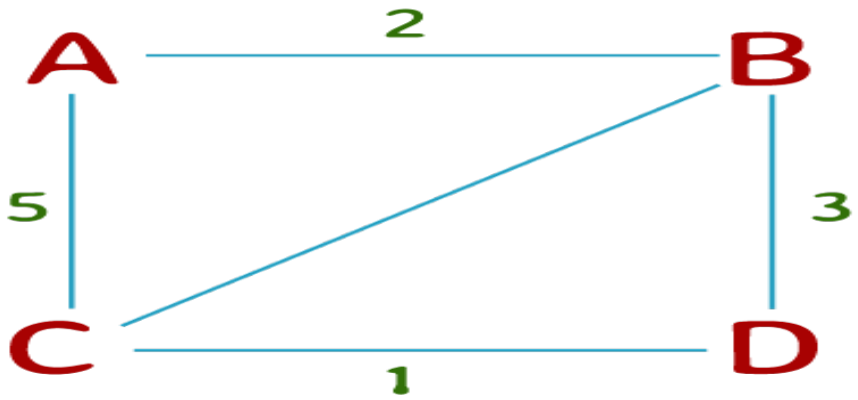


Figure: A Weighted Graph

In this graph, the vertices are represented by letters (A, B, C, D), and the numbers on the edges represent the weights of those edges.

To follow the Floyd-Warshall algorithm to this graph, we start by way of initializing a matrix of distances among every pair of vertices. If two vertices are immediately related by using a side, their distance is the load of that edge. If there may be no direct edge among vertices, their distance is infinite.

	A	**B**	**C**	**D**
A	0	2	inf	5
B	inf	0	3	2
C	5	inf	0	1
D	inf	inf	inf	0

In the first iteration of the set of rules, we keep in mind the possibility of the usage of vertex 1 (A) as an intermediate vertex in paths among all pairs of

vertices. If the space from vertex 1 to vertex 2 plus the space from vertex 2 to vertex three is much less than the present-day distance from vertex 1 to vertex three, then we replace the matrix with this new distance. We try this for each possible pair of vertices.

	A	B	C	D
A	0	2	5	4
B	inf	0	3	2
C	5	7	0	1
D	inf	inf	inf	0

In the second iteration, we recollect the possibility to use of vertex 2 (B) as an intermediate vertex in paths among all pairs of vertices. We replace the matrix in the same manner as earlier before.

	A	B	C	D
A	0	2	5	4
B	inf	0	3	2
C	5	7	0	1
D	inf	2	5	0

In the third iteration, we consider the possibility of using vertex 3 (C) as an intermediate vertex in paths between all pairs of vertices.

	A	B	C	D
A	0	2	5	4
B	inf	0	3	2
C	5	7	0	1
D	6	2	5	0

Finally, in the fourth and final iteration, we consider the possibility of using vertex 4 (D) as an intermediate vertex in paths between all pairs of vertices.

	A	B	C	D
A	0	2	5	4
B	7	0	3	2
C	5	7	0	1
D	6	2	5	0

After the fourth iteration, we have got the shortest path between every pair of vertices in the graph. For example, the shortest path from vertex A to vertex D is 4, which is the value in the matrix at row A and column D.

After the fourth iteration, we have got the shortest path between every pair of vertices in the graph. For example, the shortest path from vertex A to vertex D is 4, which is the value in the matrix at row A and column D.

Understanding Time and Space Complexities of Floyd-Warshall Algorithm:

1. Time complexity - O(n^3)
2. Space complexity - O(n^2)

Time complexity - This algorithm has an overall time complexity of O(N^3), in which N is the wide variety of vertices within the graph. It is suitable for dense graphs with poor weight cycles, as it can manage them without getting caught in a limitless loop. However, for sparse graphs, different algorithms like Dijkstra's algorithm or Bellman-Ford algorithms can be extra efficient.

Let's complete our unfinished discussion of why the Bellman-Ford or Dijkstra's shortest path algorithm is not a suitable option for solving the shortest path problem including all pairs of nodes while we are talking about time complexity.

The Bellman-Ford and Dijkstra's shortest path algorithms only calculate the shortest path from one node to all other nodes, and their upper bounds, where nn and mm are the numbers of nodes and edges, respectively, are O(nm) and O(n+mlogn).

Additionally, Dijkstra's shortest path algorithm is ineffective for negative weighted edges.

If the Bellman-Ford algorithm is expanded to include all nodes, the time complexity changes to n*O(nm) = O(n^2*m).

The Floyd Warshall algorithm only considers the number of nodes, hence the maximum number of edges for a directed network can be as high as n*(n-1), meaning that the overall complexity could be as high as O(n^4).

As a result, the Floyd-Warshall algorithm is a better option!

Space Complexity - The algorithm requires a two-dimensional array, normally called a "distance matrix" to store the shortest path distances among every pair of vertices in the graph. The size of the space matrix is decided by way of the variety of vertices inside the graph, and it has a space complexity of O(V^2), wherein V is the range of vertices within the graph.

In addition to the space matrix, the algorithm additionally calls for a two-dimensional array to hold track of the intermediate vertices on the shortest path among any two vertices. This array is normally known as the "path matrix"; and it also has a space complexity of O(V^2).

Therefore, the general space complexity of the Floyd-Warshall set of rules is O(V^2), which means that the quantity of memory required through the algorithm grows quadratically with the variety of vertices inside the graph. This can be a problem for big graphs with many vertices, because the algorithm can fast turn out to be memory-extensive and won't be viable to run on machines with restricted memory.

Some Advantages and Disadvantages of Floyd-Warshall Algorithm:

Let us now discuss some advantages and disadvantages of using Floyd-Warshall Algorithm:

Advantages of the Floyd-Warshall algorithm include:

1. It can discover the shortest direction between all pairs of vertices in a weighted graph, such as graphs with negative edge weights.
2. It is an easy and smooth-to-put algorithm, making it accessible to developers of all skill ranges.
3. It is appropriate for both dense and sparse graphs.
4. It has a time complexity of O(N^3), that is relatively efficient for most real-international applications.
5. It can be used to discover negative weight cycles in a graph.

Disadvantages of the Floyd-Warshall set of rules include:

1. It calls for a matrix of size N^2 to store the intermediate results, which may be prohibitively large for extremely large graphs.
2. It is not the maximum green set of rules for fixing the all-pairs shortest path hassle in sure types of graphs, inclusive of sparse graphs or graphs with non-bad part weights.
3. It won't be suitable for real-time packages or packages with strict reminiscence constraints, as it is able to take a long term to compute the shortest paths in very huge graphs.
4. It can be less intuitive than different algorithms, which include Dijkstra's algorithm, or the Bellman-Ford set of rules, making it more difficult to understand for some builders.

Applications of Floyd Warshall Algorithm:

The Floyd-Warshall algorithm is a dynamic programming algorithm used for finding the shortest course among all pairs of vertices in a weighted graph. Here are some of the programs of the Floyd-Warshall algorithm:

1. **Routing Algorithms:** The Floyd-Warshall algorithm is broadly utilized in routing algorithms, along with within the OSPF (Open Shortest Path First) protocol utilized in Internet routing. It can help decide the shortest route among two nodes in a network and is useful in locating the least congested path.
2. **Airline Networks:** The Floyd-Warshall set of rules can also be utilized in airline networks to locate the shortest path between two cities with the lowest cost. It can assist airways plan their routes and limit fuel charges.
3. **Traffic Networks:** The algorithm is used to find the shortest path between points in a visitors' network. It can help reduce congestion and improve the go with the flow of visitors in city areas.

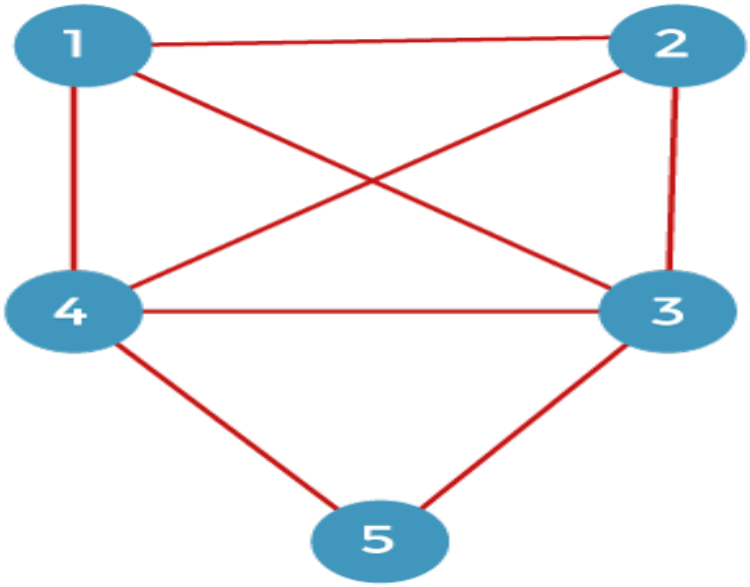
4. **Computer Networks:** The Floyd-Warshall algorithm is likewise utilized in laptop networks to decide the shortest course between hosts in a network. It can assist in minimizing community latency and improve community overall performance.
5. **Game Development:** The set of rules may be used in game development to find the shortest direction among two items in a sport world. It is beneficial in games in which the participant desires to navigate through complex surroundings, together with a maze or a metropolis.

These are only some of the numerous applications of the Floyd-Warshall algorithm. It is a powerful algorithm that can be applied in an extensive variety of fields to remedy a whole lot of troubles.

Lecture 4

Traveling Salesperson problem using branch and bound

Given the vertices, the problem here is that we have to travel each vertex exactly once and reach back to the starting point. Consider the below graph:



As we can observe in the above graph that there are 5 vertices given in the graph. We have to find the shortest path that goes through all the vertices once and returns back to the starting vertex. We mainly consider the starting vertex as 1, then traverse through the vertices 2, 3, 4, and 5, and finally return to vertex 1.

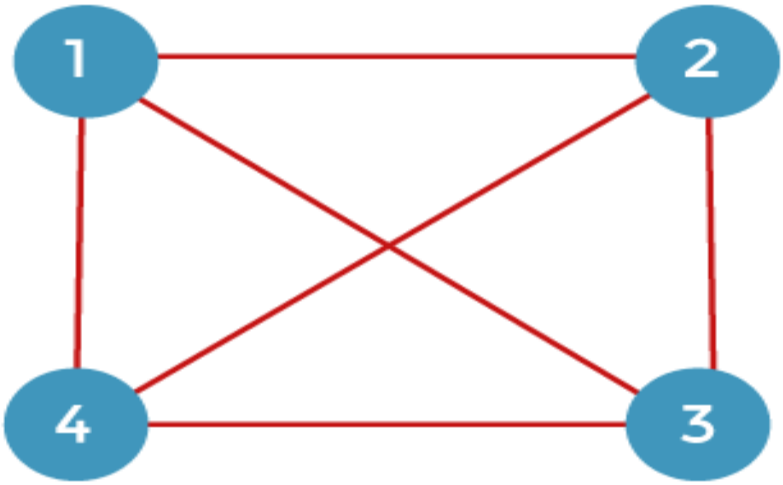
The adjacent matrix of the problem is given below:

	1	2	3	4	5
1	∞	20	30	10	11
2	15	∞	30	10	11
3	3	5	∞	2	4
4	19	6	18	∞	3
5	16	4	7	16	∞

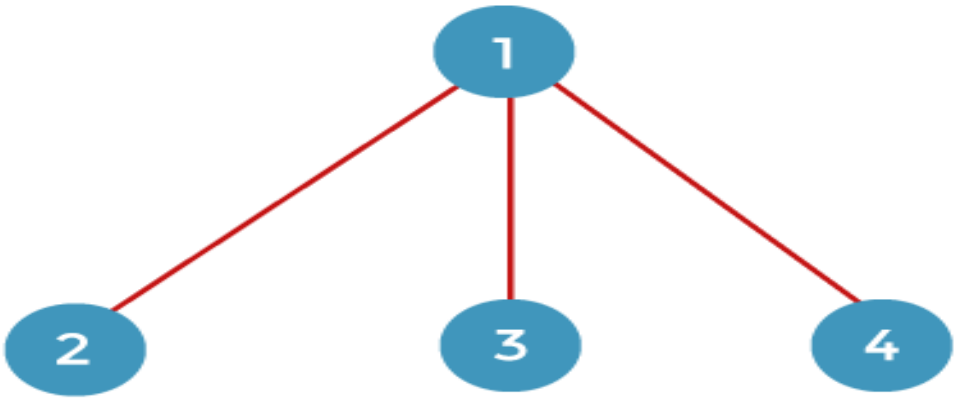
Now we look at how this problem can be solved using the branch n bound.

Let's first understand the approach then we solve the above problem.

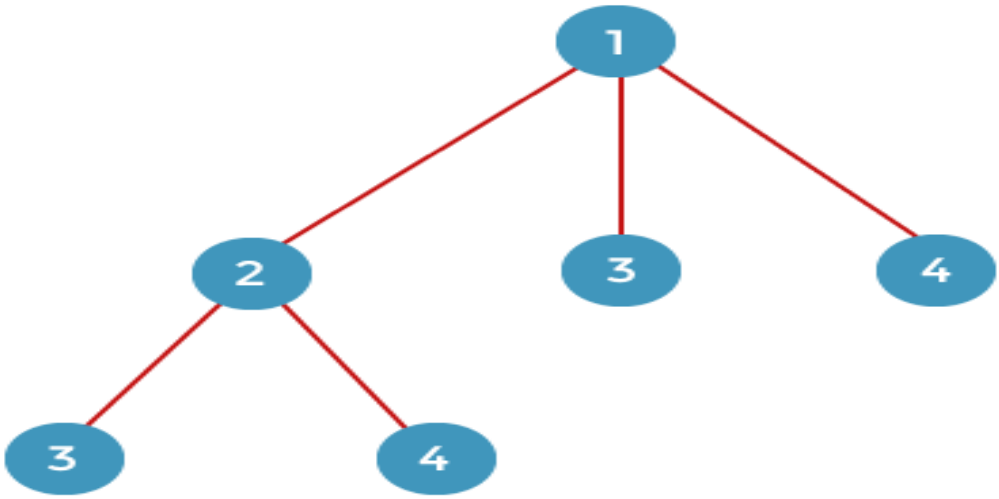
The graph is given below, which has four vertices:



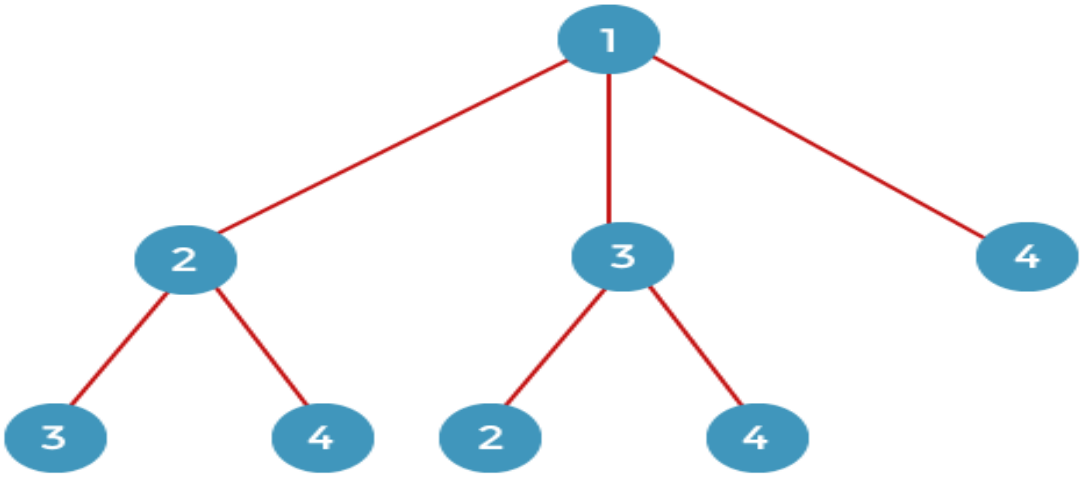
Suppose we start travelling from vertex 1 and return back to vertex 1. There are various ways to travel through all the vertices and returns to vertex 1. We require some tools that can be used to minimize the overall cost. To solve this problem, we make a state space tree. From the starting vertex 1, we can go to either vertices 2, 3, or 4, as shown in the below diagram.



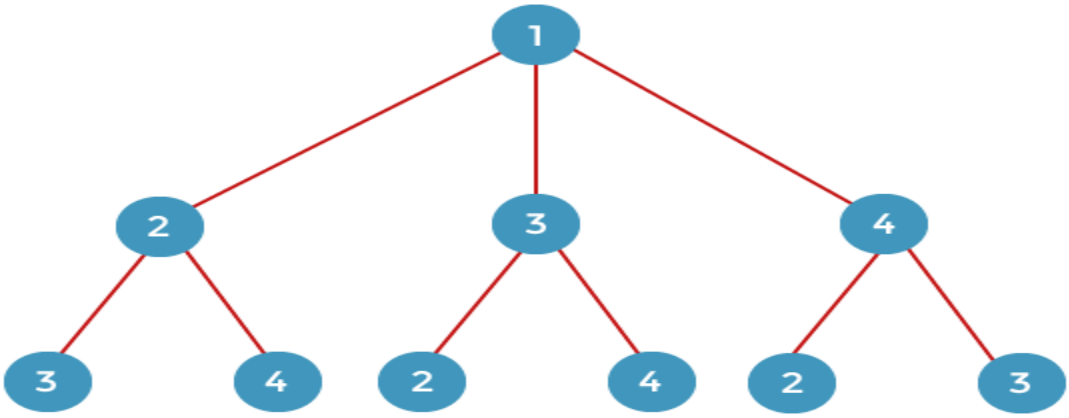
From vertex 2, we can go either to vertex 3 or 4. If we consider vertex 3, we move to the remaining vertex, i.e., 4. If we consider the vertex 4 shown in the below diagram:



From vertex 3, we can go to the remaining vertices, i.e., 2 or 4. If we consider the vertex 2, then we move to remaining vertex 4, and if we consider the vertex 4 then we move to the remaining vertex, i.e., 3 shown in the below diagram:

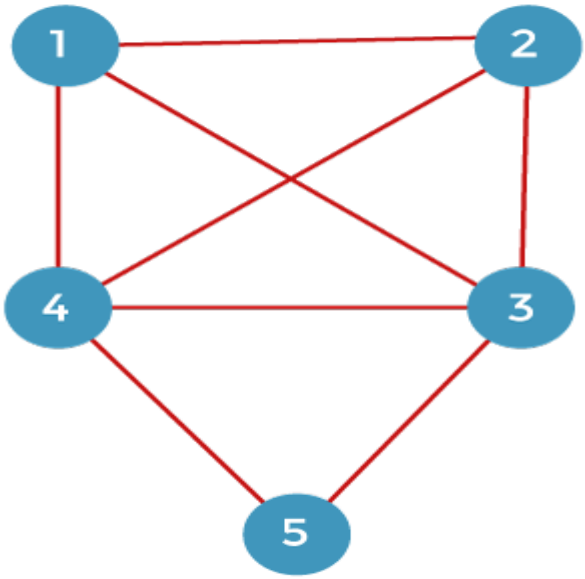


From vertex 4, we can go to the remaining vertices, i.e., 2 or 3. If we consider vertex 2, then we move to the remaining vertex, i.e., 3, and if we consider the vertex 3, then we move to the remaining vertex, i.e., 2 shown in the below diagram:



The above is the complete state space tree. The state space tree shows all the possibilities. Backtracking and branch n bound both use the state space tree, but their approach to solve the problem is different. Branch n bound is a better approach than backtracking as it is more efficient. In order to solve the problem using branch n bound, we use a level order. First, we will observe in which order, the nodes are generated. While creating the node, we will calculate the cost of the node simultaneously. If we find the cost of any node greater than the upper bound, we will remove that node. So, in this case, we will generate only useful nodes but not all the nodes.

Let's consider the above problem.





As we can observe in the above adjacent matrix that 10 is the minimum value in the first row, 2 is the minimum value in the second row, 2 is the minimum value in the third row, 3 is the minimum value in the third row, 3 is the minimum value in the fourth row, and 4 is the minimum value in the fifth row.

Now, we will reduce the matrix. We will subtract the minimum value with all the elements of a row. First, we evaluate the first row. Let's assume two variables, i.e., i and j, where 'i' represents the rows, and 'j' represents the columns.

When i = 0, j =0

M[0][0] = ∞-10= ∞

When i = 0, j = 1

M[0][1] = 20 - 10 = 10

When i = 0, j = 2

M[0][2] = 30 - 10 = 20

When i = 0, j = 3

M[0][3] = 10 - 10 = 0

When i = 0, j = 4

M[0][4] = 11 - 10 = 1

The matrix is shown below after the evaluation of the first row:

	0	1	2	3	4
0	∞	10	20	0	1
1	13	∞	14	2	0
2	3	5	∞	2	4
3	19	6	18	∞	3
4	16	4	7	16	∞

Consider the second row.

When i = 1, j =0

M[1][0] = 15-2= 13

When i = 1, j = 1

M[1][1] = ∞ - 2= ∞

When i = 1, j = 2

M[1][2] = 16 - 2 = 14

When i = 1, j = 3

M[1][3] = 4 - 2 = 2

When i = 1, j = 4

M[1][4] = 2 - 2 = 0

The matrix is shown below after the evaluation of the second row:

	0	1	2	3	4
0	∞	10	20	0	1
1	13	∞	14	2	0
2	1	3	∞	0	2
3	19	6	18	∞	3
4	16	4	7	16	∞

Consider the third row:

When i = 2, j =0

M[2][0] = 3-2= 1

When i = 2, j = 1

M[2][1] = 5 - 2= 3

When i = 2, j = 2

M[2][2] = ∞ - 2 = ∞

When i = 2, j = 3

M[2][3] = 2 - 2 = 0

When i = 2, j = 4

M[2][4] = 4 - 2 = 2

The matrix is shown below after the evaluation of the third row:

Consider the fourth row:

When i = 3, j =0

M[3][0] = 19-3= 16

When i = 3, j = 1

M[3][1] = 6 - 3= 3

When i = 3, j = 2

M[3][2] = 18 - 3 = 15

When i = 3, j = 3

M[3][3] = ∞ - 3 = ∞

When i = 3, j = 4

M[3][4] = 3 - 3 = 0

The matrix is shown below after the evaluation of the fourth row:

	0	1	2	3	4
0	∞	10	20	0	1
1	13	∞	14	2	0
2	1	3	∞	0	2
3	16	3	15	∞	0
4	16	4	7	16	∞

Consider the fifth row:

When i = 4, j =0

M[4][0] = 16-4= 12

When i = 4, j = 1

M[4][1] = 4 - 4= 0

When i = 4, j = 2

M[4][2] = 7 - 4 = 3

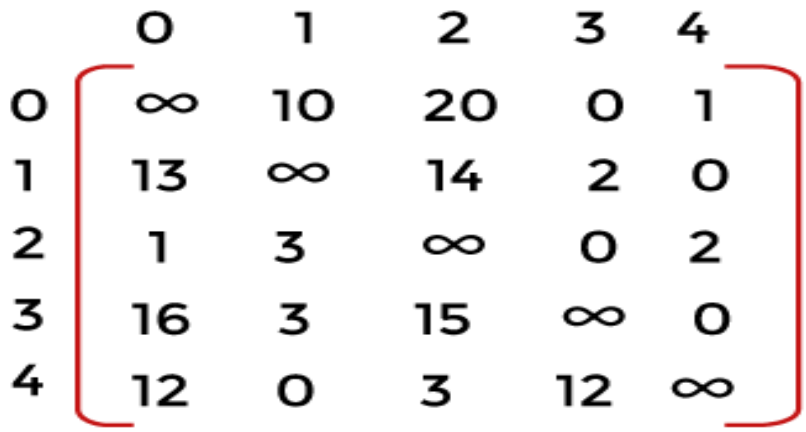
When i = 4, j = 3

M[4][3] = 16 - 4 = 12

When i = 4, j = 4

M[4][4] = ∞ - 4 = ∞

The matrix is shown below after the evaluation of the fifth row:



	0	1	2	3	4
0	∞	10	20	0	1
1	13	∞	14	2	0
2	1	3	∞	0	2
3	16	3	15	∞	0
4	12	0	3	12	∞

The above matrix is the reduced matrix with respect to the rows.

Now we reduce the matrix with respect to the columns. Before reducing the matrix, we first find the minimum value of all the columns. The minimum value of first column is 1, the minimum value of the second column is 0, the minimum value of the third column is 3, the minimum value of the fourth column is 0, and the minimum value of the fifth column is 0, as shown in the below matrix:

Now we reduce the matrix.

Consider the first column.

When i = 0, j =0

M[0][0] = ∞-1= ∞

When i = 1, j = 0

M[1][0] = 13 - 1= 12

When i = 2, j = 0

M[2][0] = 1 - 1 = 0

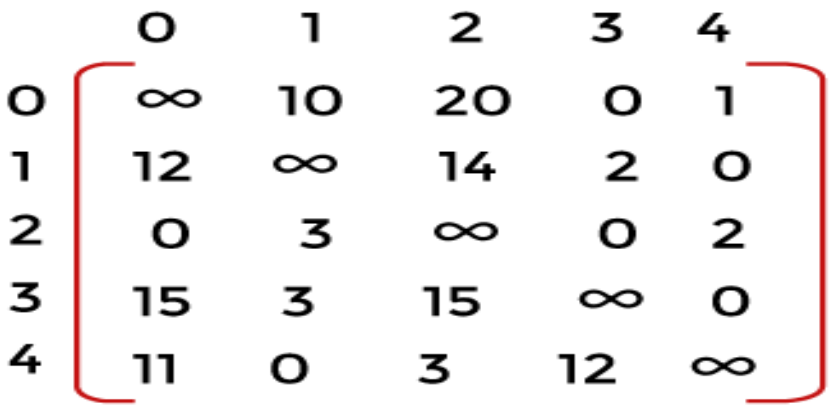
When i = 3, j = 0

M[3][0] = 16 - 1 = 15

When i = 4, j = 0

M[4][0] = 12 - 1 = 11

The matrix is shown below after the evaluation of the first column:



	0	1	2	3	4
0	∞	10	20	0	1
1	12	∞	14	2	0
2	0	3	∞	0	2
3	15	3	15	∞	0
4	11	0	3	12	∞

Since the minimum value of the first and the third columns is non-zero, we will evaluate only first and third columns. We have evaluated the first column. Now we will evaluate the third column.

Consider the third column.

When i = 0, j =2

M[0][2] = 20-3= 17

When i = 1, j = 2

M[1][2] = 13 - 1= 12

When i = 2, j = 2

M[2][2] = 1 - 1 = 0

When i = 3, j = 2

M[3][2] = 16 - 1 = 15

When i = 4, j = 2

M[4][2] = 12 - 1 = 11

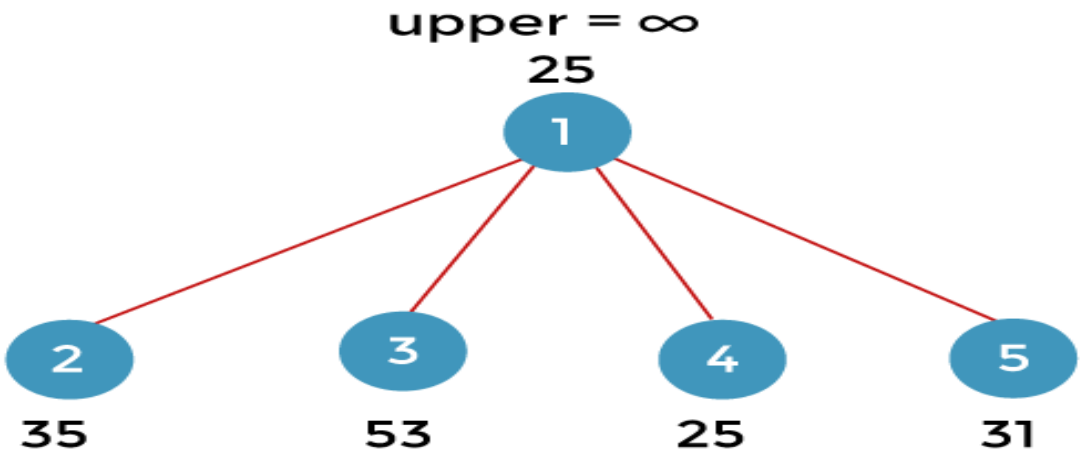
The matrix is shown below after the evaluation of the third column:

	0	1	2	3	4
0	∞	10	17	0	1
1	12	∞	12	2	0
2	0	3	0	0	2
3	15	3	15	∞	0
4	11	0	0	12	∞

The above is the reduced matrix. The minimum value of rows is 21, and the columns is 4. Therefore, the total minimum value is (21 + 4) equals to 25.

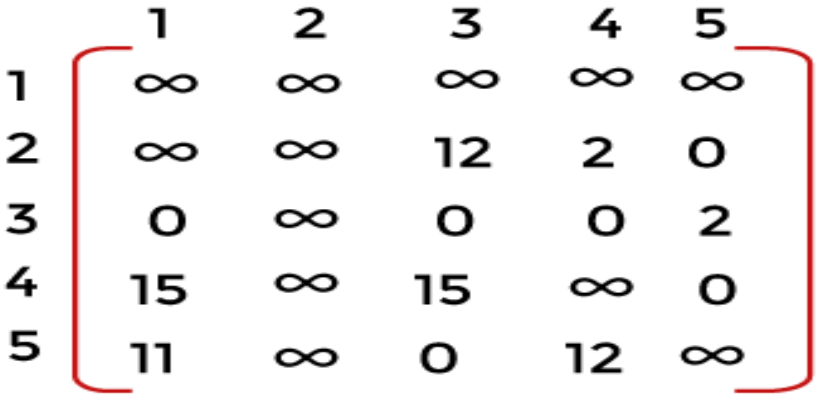
Let's understand that how to solve this problem using branch and bound with the help of a state-space tree.

To make a state-space tree, first, we consider node 1. From node 1, we can go either to nodes 2, 3, 4, or 5 as shown in the below image. The cost of node 1 would be the cost which we achieved in the above-reduced matrix, i.e.., 25. Here, we will also maintain the upper bound. Initially, the upper bound would-be infinity.



Now, consider node 2. It means that we are moving from node 1 to node 2. Make the first row and second column as infinity shown in the below matrix:

Once we move from node 1 to node 2, we cannot move back to node 1. Therefore, we have to make 2 to 1 as infinity shown in the below matrix:



	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	∞	∞	12	2	0
3	0	∞	0	0	2
4	15	∞	15	∞	0
5	11	∞	0	12	∞

Since each row and column contains atleast one zero value; therefore, we can say that above matrix has been reduced. The cost of reduction of node 2 is c(1, 2) + r + r` = 10 + 25 + 0 = 35.

Now we will find the minimum value of each column of the new reduced matrix. The minimum value of the first column is 11 and the minimum value of other three columns is 0.

Now, consider the node 3. It means that we are moving from the node 1 to node 3. Make the first row and third column as infinity shown in the below matrix:

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	12	∞	∞	2	0
3	0	3	∞	0	2
4	15	3	∞	∞	0
5	11	0	∞	12	∞

Once we move from the node 1 to node 3, we cannot move back to the node 1. Therefore, we have to make 3 to 1 as infinity shown in the below matrix:

Since each row and column contains atleast one zero value; therefore, we can say that above matrix has been reduced. The cost of reduction of node 3 is c(1, 3) + r + r` = 17 + 25 + 11= 53.

Now, consider the node 4. It means that we are moving from the node 1 to node 4. Make the first row and forth column as infinity shown in the below matrix:

	1	2	3	4	5
1	∞	∞	∞	∞	∞
2	12	∞	12	∞	0
3	0	∞	0	∞	2
4	∞	∞	∞	∞	∞
5	11	∞	0	∞	∞

Once we move from the node 1 to node 4, we cannot move back to the node 1. Therefore, we have to make 4 to 1 as infinity shown in the below matrix:

Since each row and column contains atleast one zero value; therefore, we can say that above matrix has been reduced. The cost of reduction of node 4 is c(1, 4) + r + r` = 0 + 25 + 0 = 25.

Now, consider the node 5. It means that we are moving from the node 1 to node 5. Make the first row and fifth column as infinity shown in the below matrix:

Once we move from the node 1 to node 5, we cannot move back to the node 1. Therefore, we have to make 5 to 1 as infinity shown in the below matrix:

Since each row and column contains atleast one zero value; therefore, we can say that above matrix has been reduced. In this case, second and third rows are non-zero. Therefore, we have to first find the minimum values of both the rows. The minimum value of second row is 2; therefore, we subtract 2 from all the elements of the second row. The elements of second row would be:

A[1][0] = 12-2 = 10

A[1][1] = ∞

A[1][2] = 11 - 2 = 9

A[1][3] = 2 - 2 = 0

A[1][4] = ∞ - 2 = ∞

As we can observe now that the second row contains one zero value.

The cost of reduction of node 5 is c(1, 5) + r + r` = 1 + 25 + 5 = 31

Since the node 4 has the minimum cost, i.e., 25. So we will explore the node 4 first. From the vertex 4, we can go either to the vertex 2, 3 or 5 as shown in the below image:

Now we have to calculate the cost of the path from the vertex 4 to 2, vertex 4 to 3, and vertex 4 to 5. Here, we will use the matrix of node 4 to find the cost of all the nodes.

First, we consider the path from the vertex 4 to the vertex 2. We make fourth row as ∞ and second column as ∞. Since we cannot move back from 2 to 1, so 1 to 2 is also infinity as shown in the below matrix:

Since all the rows and columns have atleast one zero value. Therefore, we can say that this matrix is already reduced. So, there would be no reduction cost. The cost of reduction of node 2 is c(4, 2) + r + r` = 3 + 25 + 0 = 28

Now we have to calculate the cost of the path from the vertex 4 to the vertex 3. We make fourth row and third column as infinity as shown in the below matrix. Since we cannot move from the vertex 3 to 1, so we make 3 to 1 as infinity shown in the below matrix:

Now we will check whether each row and column contain atleast one zero value or not. First, we observe all the rows. Since the third row does not have a zero value, so we first find the minimum value of the third row. The minimum value of the third row is 2, so we subtract 2 from all the elements of the third row. The elements of third row would be:

A[2][0] = ∞ - 2 = ∞

A[2][1] = 3 - 2 = 1

A[2][2] = ∞ - 2 = ∞

A[2][3] = ∞ - 2 = ∞

A[2][4] = 2 - 2 = 0

As we can observe now that the third row contains one zero value.

The first column does not contain the zero value. The minimum value of the first column is 11. We subtract 11 from all the elements of the first column. The elements of first column would be:

A[0][0] = ∞ - 11 = ∞

A[1][0] = 12 - 11 = 1

A[2][0] = ∞ - 11= ∞

A[3][0] = ∞ - 11= ∞

A[4][0] = 11 - 11 = 0

As we can observe now that the first column contains one zero value. The total minimum cost is 11 +2 equals to 13. The cost of reduction of node 3 is c(4, 3) + r + r` = 12 + 25 + 13 = 50.

Now we will calculate the cost of the path from the vertex 4 to 5. We make fourth row and fifth column as infinity. Since we cannot move back from the node 5 to 1, so we also make 1 to 5 as infinity shown in the below matrix:

Now we will check whether each row and column contain atleast one zero value or not. First, we observe all the rows. The second row does not contain the zero value, so we find the minimum value of the second row. The minimum value is 11 so we subtract 11 from all the elements of the second row. The elements of second row would be:

A[1][0] = 12 - 11 = 1

A[1][1] = ∞ - 11 = ∞

A[1][2] = 11 - 11 = 0

A[1][3] = ∞ - 11 = ∞

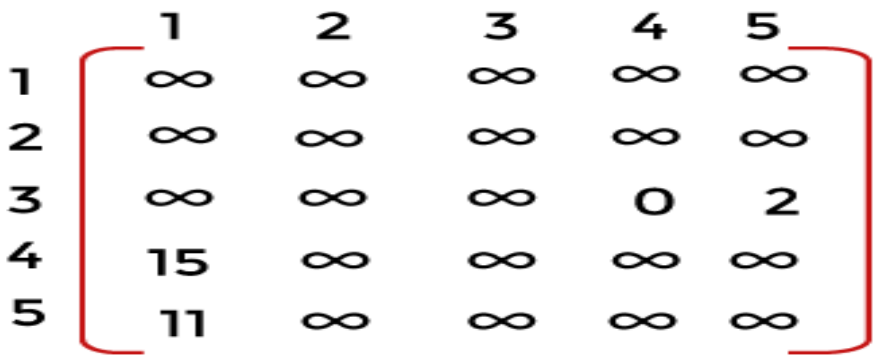
A[1][4] = ∞ - 11 = ∞

As we can observe now that the second row contains one zero value. The cost of reduction of node 5 is c(4, 5) + r + r` = 0 + 25 + 11 = 36.

Now we will compare the cost of all the leaf nodes. The node with a cost 28 is minimum so we will explore this node. The node with a cost 28 can be further expanded to the nodes 3 and 5 as shown in the below figure:

Now we have to calculate the cost of both the nodes, i.e., 3 and 5. First we consider the path from node 2 to node 3. Consider the matrix of node 2 which is shown below:

We make second row and third column as infinity. Also, we cannot move back from the node 3 to node 1 so we make 3 to 1 as infinity as shown in the below matrix:



Now we will check whether any row contains zero value or not. Since third row does not contain any zero value so we will find the minimum value of the third row. The minimum value of the third row is 2 so we subtract 2 from all the elements of the third row. The elements of third row would be:

A[2][0] = ∞ - 2 = ∞

A[2][1] = ∞ - 2 = ∞

A[2][2] = ∞ - 2 = ∞

A[2][3] = ∞ - 2 = ∞

A[2][4] = 2 - 2 = 0

Since fifth row does not contain any zero value so we will find the minimum value of the fifth row. The minimum value of the fifth row is 11 so we subtract 11 from all the elements of the fifth row.

A[4][0] = 11 - 11 = 0

A[4][1] = ∞ - 11 = ∞

A[4][2] = ∞ - 11 = ∞

A[4][3] = ∞ - 11 = ∞

A[4][4] = ∞ - 11 = ∞

The total minimum cost is (11 + 2) equals to 13. The cost of reduction of node 3 is c(2, 3) + r + r` = 11 + 28 + 13 = 52.

Consider the path from node 2 to node 5. Make the fourth row and third column as infinity. Since we cannot move back from the node 5 to node 1 so make 1 to 5 also as infinity as shown in the below matrix:

Now we will check whether any row contains zero value or not. Since every row and column contains a zero value; therefore, the above matrix is the reduced matrix.

The cost of reduction of node 5 is c(2, 5) + r + r` = 0 + 28 + 0 = 28

Now we will find out the leaf node with a minimum cost. The node 5 with a cost 28 is minimum so we select node 5 for the further exploration. The node 5 can be further expanded to the node 3 as shown in the below figure:

Here, we will use the matrix of node 5 having cost 28 as shown below:

Consider the path from node 5 to node 3. Make the fifth row and third column as infinity. Since we cannot move back from the node 3 to node 1 so make 1 to 5 also as infinity as shown in the below matrix:

Now we will check whether any row contains zero value or not. Since every row and column contains a zero value; therefore, the above matrix is the reduced matrix.

The cost of reduction of node 3 is c(5, 3) + r + r` = 0 + 28 + 0 = 28.

Finally, we traverse all the nodes. The upper value is updated from infinity to 28. We will check whether any leaf node has a value less than 28. Since no leaf node contains the value less than 28 so we discard all the leaf nodes from the tree as shown below:

The path of the tour would be 1->4->2->5->3.

Lecture 5

Longest Common Subsequence

Here longest means that the subsequence should be the biggest one. The common means that some of the characters are common between the two strings. The subsequence means that some of the characters are taken from the string that is written in increasing order to form a subsequence.

Let's understand the subsequence through an example.

Suppose we have a string 'w'.

W_1 = abcd

The following are the subsequences that can be created from the above string:

- ab
- bd
- ac
- ad
- acd
- bcd

The above are the subsequences as all the characters in a sub-string are written in increasing order with respect to their position. If we write ca or da then it would be a wrong subsequence as characters are not appearing in the increasing order. The total number of subsequences that would be possible is 2^n, where n is the number of characters in a string. In the above string, the value of 'n' is 4 so the total number of subsequences would be 16.

W_2= bcd

By simply looking at both the strings w1 and w2, we can say that bcd is the longest common subsequence. If the strings are long, then it won't be possible to find the subsequence of both the string and compare them to find the longest common subsequence.

Finding LCS using dynamic programming with the help of a table.

Consider two strings:

X= a b a a b a

Y= b a b b a b

(a, b)

For index i=1, j=1

	^	b	a	b	b	a	b
^	0	0	0	0	0	0	0
a	0	0					
b	0						
a	0						
a	0						
b	0						
a	0						

Since both the characters are different so we consider the maximum value. Both contain the same value, i.e., 0 so put 0 in (a,b). Suppose we are taking the 0 value from 'X' string, so we put arrow towards 'a' as shown in the above table.

(a, a)

For index i=1, j=2

	^	b	a	b	b	a	b
^	0	0	0	0	0	0	0
a	0	0	1				
b	0						
a	0						
a	0						
b	0						
a	0						

Both the characters are the same, so the value would be calculated by adding 1 and upper diagonal value. Here, upper diagonal value is 0, so the value of this entry would be (1+0) equal to 1. Here, we are considering the upper diagonal value, so the arrow will point diagonally.

(a, b)

For index i=1, j=3

	^	b	a	b	b	a	b
^	0	0	0	0	0	0	0
a	0	0	1	1			
b	0						
a	0						
a	0						
b	0						
a	0						

Since both the characters are different so we consider the maximum value. The character 'a' has the maximum value, i.e., 1. The new entry, i.e., (a, b) will contain the value 1 pointing to the 1 value.

(a, b)

For index i=1, j=4

	^	b	a	b	b	a	b
^	0	0	0	0	0	0	0
a	0	0	1	1	1		
b	0						
a	0						
a	0						
b	0						
a	0						

Since both the characters are different so we consider the maximum value. The character 'a' has the maximum value, i.e., 1. The new entry, i.e., (a, b) will contain the value 1 pointing to the 1 value.

(a, a)

For index i=1, j=5

	^	b	a	b	b	a	b
^	0	0	0	0	0	0	0
a	0	0	1	1	1	1	
b	0						
a	0						
a	0						
b	0						
a	0						

Both the characters are same so the value would be calculated by adding 1 and upper diagonal value. Here, upper diagonal value is 0 so the value of this entry would be (1+0) equal to 1. Here, we are considering the upper diagonal value so arrow will point diagonally.

(a, b)

For index i=1, j=6

	^	b	a	b	b	a	b
^	0	0	0	0	0	0	0
a	0	0	1	←1	←1	←1	←1
b	0						
a	0						
a	0						
b	0						
a	0						

Since both the characters are different so we consider the maximum value. The character 'a' has the maximum value, i.e., 1. The new entry, i.e., (a, b) will contain the value 1 pointing to the 1 value.

(b, b)

For index i=2, j=1

	^	b	a	b	b	a	b
^	0	0	0	0	0	0	0
a	0	0	1	←1	←1	1	←1
b	0	1					
a	0						
a	0						
b	0						
a	0						

Both the characters are same so the value would be calculated by adding 1 and upper diagonal value. Here, upper diagonal value is 0 so the value of this entry would be (1+0) equal to 1. Here, we are considering the upper diagonal value so arrow will point diagonally.

(b, a)

For index i=2, j=2

	^	b	a	b	b	a	b
^	0	0	0	0	0	0	0
a	0	0	1	←1	←1	←1	←1
b	0	↖1	←1				
a	0						
a	0						
b	0						
a	0						

Since both the characters are different so we consider the maximum value. The character 'a' has the maximum value, i.e., 1. The new entry, i.e., (a, b) will contain the value 1 pointing to the 1 value.

In this way, we will find the complete table. The final table would be:

	^	b	a	b	b	a	b
^	0	0	0	0	0	0	0
a	0	0	↖1	←1	←1	←1	←1
b	0	↖1	←1	↖2	↖2	←2	↖2
a	0	↑1	↖2	←2	←2	↖3	← 3
a	0	↑ 1	↖2	←2	←2	↖3	← 3
b	0	↖1	←1	↖3	↖3	←3	↖ 4
a	0	↑1	↖2	←2	←2	↖4	←4

In the above table, we can observe that all the entries are filled. Now we are at the last cell having 4 value. This cell moves at the left which contains 4 value.; therefore, the first character of the LCS is 'a'.

The left cell moves upwards diagonally whose value is 3; therefore, the next character is 'b' and it becomes 'ba'. Now the cell has 2 value that moves on the left. The next cell also has 2 value which is moving upwards; therefore, the next character is 'a' and it becomes 'aba'.

The next cell is having a value 1 that moves upwards. Now we reach the cell (b, b) having value which is moving diagonally upwards; therefore, the next character is 'b'. The final string of longest common subsequence is 'baba'.

Why a dynamic programming approach in solving a LCS problem is more efficient than the recursive algorithm?

If we use the dynamic programming approach, then the number of function calls are reduced. The dynamic programming approach stores the result of each

function call so that the result of function calls can be used in the future function calls without the need of calling the functions again.

In the above dynamic algorithm, the results obtained from the comparison between the elements of x and the elements of y are stored in the table so that the results can be stored for the future computations.

The time taken by the dynamic programming approach to complete a table is O(mn) and the time taken by the recursive algorithm is $2^{max(m, n)}$.

Algorithm of Longest Common Subsequence

1. Suppose X and Y are the two given sequences
2. Initialize a table of LCS having a dimension of X.length * Y.length
3. XX.label = X
4. YY.label = Y
5. LCS[0][] = 0
6. LCS[][0] = 0
7. Loop starts from the LCS[1][1]
8. Now we will compare X[i] and Y[j]
9. if X[i] is equal to Y[j] then
10. LCS[i][j] = 1 + LCS[i-1][j-1]
11. Point an arrow LCS[i][j]
12. Else
13. LCS[i][j] = max(LCS[i-1][j], LCS[i][j-1])

Lecture 6

8 queen problem

- The eight queens problem is the problem of placing eight queens on an 8×8 chessboard such that none of them attack one another (no two are in the same row, column, or diagonal). More generally, the n queens problem places n queens on an n×n chessboard. There are different solutions for the problem. Backtracking | Set 3 (N Queen Problem) Branch and Bound | Set 5 (N Queen Problem)

Explanation:

- This **pseudocode uses a backtracking algorithm** to find a solution to the 8 Queen problem, which consists of placing 8 queens on a chessboard in such a way that no two queens threaten each other.
- The algorithm starts by placing a queen on the first column, then it proceeds to the next column and places a queen in the **first safe** row of that column.
- If the algorithm reaches the 8th column and al**l queens are placed in a safe position**, it prints the board and returns true.
  If the algorithm is unable to place a queen in a safe position in a certain column, it backtracks to the previous column and tries a different row.
- The "isSafe" function **checks** if it is safe to place a queen on a certain row and column by checking if there are any queens in the same row, diagonal or anti-diagonal.
- It's worth to notice that this is just a high-level **pseudocode** and it might need to be adapted depending on the specific implementation and language you are using.

Knapsack problem

The Knapsack Problem is an Optimization Problem in which we have to find an optimal answer among all the possible combinations.There are three types of knapsack problems : 0-1 Knapsack, Fractional Knapsack and Unbounded Knapsack. In this article, we will discuss 0-1 Knapsack in detail.

What is 0-1 Knapsack Problem

In the 0-1 Knapsack Problem, we are given a Knapsack or a Bag that can hold weight up to a certain value. We have various items that have different weights and values associated with them. Now we have to fill the knapsack in such a way so that the sum of the total weights of the filled items does not exceed the maximum capacity of the knapsack and the sum of the values of the filled items is maximum. Refer to the diagram below for a better understanding.

Optimal Solution

Select item 2 and Item 3 which will give us total value of 140 + 60 = 200 which is the maximum value we can get among all valid combinations.

The diagram above shows a Knapsack that can hold up to a maximum weight of **30 units**. We have three items namely item1, item2, and item3. The values and weights associated with each item are shown in the diagram. Now we have to fill the knapsack in such a way so that the sum of the values of items filled in the knapsack is maximum. If we try out all the valid combinations in which the total weight of the filled items is less than or equal to 30, we will see that we can get the optimal answer by selecting item2 and item3 in the knapsack which gives us a total value of 120.

We can not fill all the items in the knapsack as the total weight of all the items will exceed the maximum weight limit of the knapsack.

Note: In the 0-1 Knapsack Problem we can either decide to keep an item in the knapsack or not keep an item in the knapsack. There is no option of partially keeping an item in the knapsack, unlike the Fractional Knapsack problem.

Lecture 7

Traveling salesman problem

Traveling-salesman Problem

In the traveling salesman Problem, a salesman must visits n cities. We can say that salesman wishes to make a tour or Hamiltonian cycle, visiting each city exactly once and finishing at the city he starts from. There is a non-negative cost c (i, j) to travel from the city i to city j. The goal is to find a tour of minimum cost. We assume that every two cities are connected. Such problems are called Traveling-salesman problem (TSP).

We can model the cities as a complete graph of n vertices, where each vertex represents a city.

It can be shown that TSP is NPC.

If we assume the cost function c satisfies the triangle inequality, then we can use the following approximate algorithm.

Triangle inequality

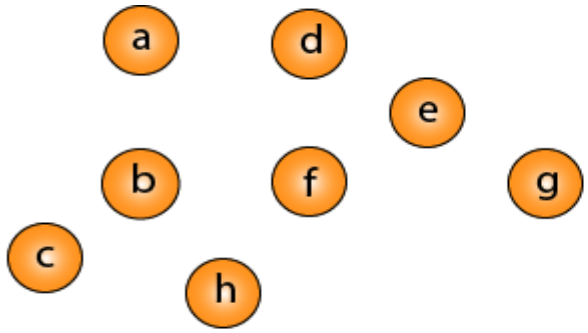
Let u, v, w be any three vertices, we have

$$c(u, w) \leq c(u, v) + c(v, w)$$

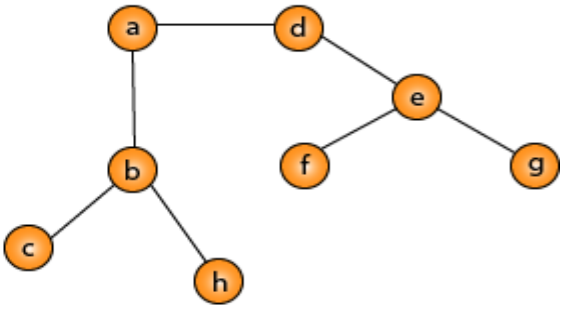
One important observation to develop an approximate solution is if we remove an edge from H*, the tour becomes a spanning tree.

1. Approx-TSP (G= (V, E))
2. {
3. 1. Compute a MST T of G;
4. 2. Select any vertex r is the root of the tree;
5. 3. Let L be the list of vertices visited in a preorder tree walk of T;
6. 4. Return the Hamiltonian cycle H that visits the vertices in the order L;
7. }

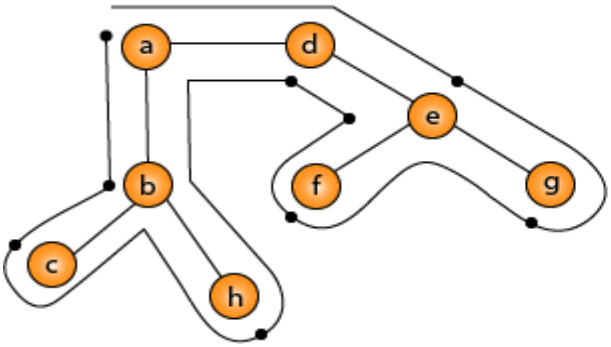
Traveling-salesman Problem



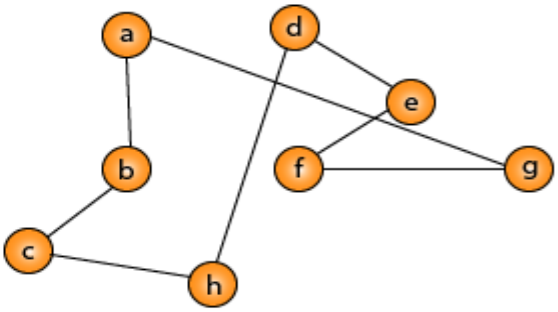
(1) A given set of points



(2) MST T



(3) Full tree walk on T.



(4) A preorder sequence gives a tour H.

Intuitively, Approx-TSP first makes a full walk of MST T, which visits each edge exactly two times. To create a Hamiltonian cycle from the full walk, it bypasses some vertices (which corresponds to making a shortcut)